SOC 跑在 ECU 上 · 电控同学操作指南

SerialAssistant 2.7 · 重庆大学方程式赛车队 E74 · 供电控同学的总操作步骤

SOC 上 ECU　125 串 / 12.041 Ah　高电压 NCM

目录

1. 1. 为什么把 SOC 放到 ECU 上跑
2. 2. 系统架构（总览）
3. 3. ECU 从 BMS 接收什么（输入数据契约）
4. 4. SOC 算法步骤（移植要点，语言无关）
5. 5. E74 参数一页纸（直接抄进代码）
6. 6. ECU 输出给 C125（输出数据契约）
7. 7. 联调与验收判据（怎么知道算对了）
8. 8. 用 SerialAssistant 当验证工具
9. 附录 A：改代码 / 调参前必读
10. 附录 B：参考文献

1. 为什么把 SOC 放到 ECU 上跑

一句话：**BMS 负责"把原始采样发出来"，SOC 计算与显示都放在 ECU（你们的主控制器）上完成。**

- 电池箱里的 `BMS_Master` 用的是 STM32F103C8T6，主要跑 isoSPI 采集、接触器保护、故障诊断这些**硬实时任务**。让它再跑 SOC 估算，既挤占算力，又要在固件里塞一张 26 点 OCV 标定表和一组实测阈值，维护成本高。
- SOC 本身逻辑不重，但它依赖一张实测的 OCV–SOC 表、一组来自真机数据的阈值，还要做"零漂在线学习""静置软锚定"这类长期校正。这些放在 **ECU** 上跑最合适：算力充裕，又能直接把算好的结果经 CAN 送给 C125 仪表盘。
- 结论：**BMS 只发原始采样（电压 / 温度 / 电流 / 接触器状态），ECU 算 SOC，C125 只显示。** 这套分工也是 SerialAssistant（PC 端参考实现）已经在跑的同一套算法。

> 本文档是给电控同学的总操作指南：ECU 要接收什么数据、算法怎么写、算完往哪发、怎么验收。所有公式和参数都来自已经在 PC 端验证过的实现，可直接移植。

---

2. 系统架构（总览）

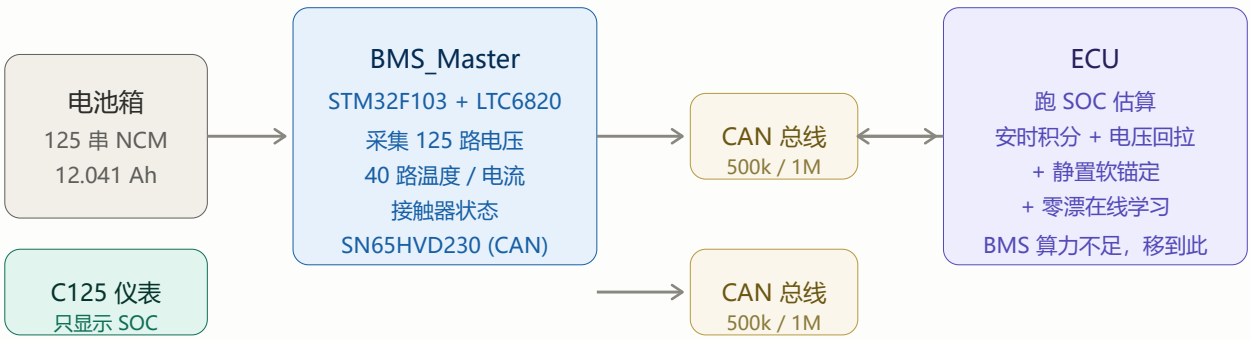


图 1：系统架构（电池箱 → BMS → CAN → ECU 算 SOC → CAN → C125）

角色	硬件	职责	不做什么
电池箱	125 串 NCM，5×25 模组	电芯本体	—
BMS_Master	STM32F103 + LTC6820 + SN65HVD230	采集 125 路电压 / 40 路温度 / 母线电流 / 接触器状态，打包成 **CAN** 帧广播	不算 SOC、不做显示
ECU	车队主控制器	收 CAN → 跑 SOC 估算 → 发出 SOC/电压/电流/工况	不碰电芯采集
C125	MoTeC 仪表盘	只显示 SOC（min/max）、总压、电流、工况	不计算

关键前提（需电控 + 电池组协调）：`BMS_Master` 板上已有 CAN 收发器（SN65HVD230），需要固件把每一轮原始采样组装成 CAN 帧广播出来。波特率建议 **500 kbps**（也支持 1M）。ECU 与 C125 之间的 CAN 同样走 500k/1M。

---

3. ECU 从 BMS 接收什么（输入数据契约）

ECU 每"一轮"必须拿到下面 **5 类信号**。一轮 = BMS 采集完 125 节电芯 + 40 路温度 + 电流 + 接触器后发出的一批帧。

#	信号	类型 / 单位	量程 / 说明
1	125 路单体端电压	uint16，**mV**	正常 2500~4600；坏点用 `0xFFFF` 或 `X` 标记
2	40 路模组温度	uint16，×0.1 °C	每模组 8 路；约 −40.0~+150.0 °C
3	母线电流	int16，×0.1 A	**放电为正**；断线/饱和时可能是 −250 A 轨值
4	接触器状态	3 bit	AIR+、AIR-、PRE；**三者全 0 = 母线断开**
5	BMS 状态	1 bit	`BMS OK` / `BMS ERROR`

3.1 真机取证带来的 4 个坑（移植时务必处理）

这些不是假设，是把 119 轮历史日志 + 499 轮会话库跑了一遍量出来的：

1. `V=` **总压传感器是坏的**：读数恒 0。所以**总压只能靠 125 节电压取均值 ×125** 估算，不能信 BMS 报的总压。
2. **电流有一段 −250 A 的平滑轨值**（约 40% 样本）。它不是噪声，是传感器饱和态；12 Ah 的包物理上不可能有 250 A。ECU 必须整帧作废这个值，不能拿去积分也不能拿去学零漂。
3. **每轮稳定约 8 个坏点**（掉字符 / 超量程 / `X`）。清洗算法要能容忍这个量级，又不能被它污染统计量。
4. `AIR+` **那一行只在约 60% 的报文块里出现**。缺了它，ECU 要把接触器状态"粘滞继承"上一次已知值（最多 10 轮），否则会拿带零漂的电流去积分，SOC 慢慢跑偏。

3.2 建议 CAN 帧布局（模板，按你们实际 CAN 矩阵调整）

> 下面是**可直接照抄的建议模板**。具体帧 ID、分帧数量由你们现有 CAN 矩阵决定；重要的是"语义契约"（上面 5 类信号、单位、量程）要对得上。

帧 ID	DLC	内容	打包方式
`0x600` + n (n=0..31)	8	单体电压	4 节/帧，每节 uint16(mV)；最后一帧不足 4 节补 `0xFFFF`
`0x640` + n (n=0..9)	8	模组温度	4 路/帧，每路 uint16(×0.1 °C)
`0x680`	8	电流 / 接触器 / 状态	byte0-1 int16 电流(×0.1A)；byte2 bit0 AIR+ / bit1 AIR- / bit2 PRE（1=闭合）；byte3 bit0 BMS OK；byte4 轮序号(0~255 循环)
轮边界	—	—	以**轮序号变化**且 125 节电压收齐为"一轮到齐"

ECU 收到 `0x680` 的轮序号变化 + 32 帧电压收集完整，即触发一次 SOC 更新。

---

4. SOC 算法步骤（移植要点，语言无关）

下面 6 步是 ECU 每收到一轮要做的。公式里的常量见第 5 章，**直接抄**。整条链路纯逻辑、无浮点黑盒，单片机上的 `double` / `float` 都够用（单轮耗时 < 0.1 ms）。

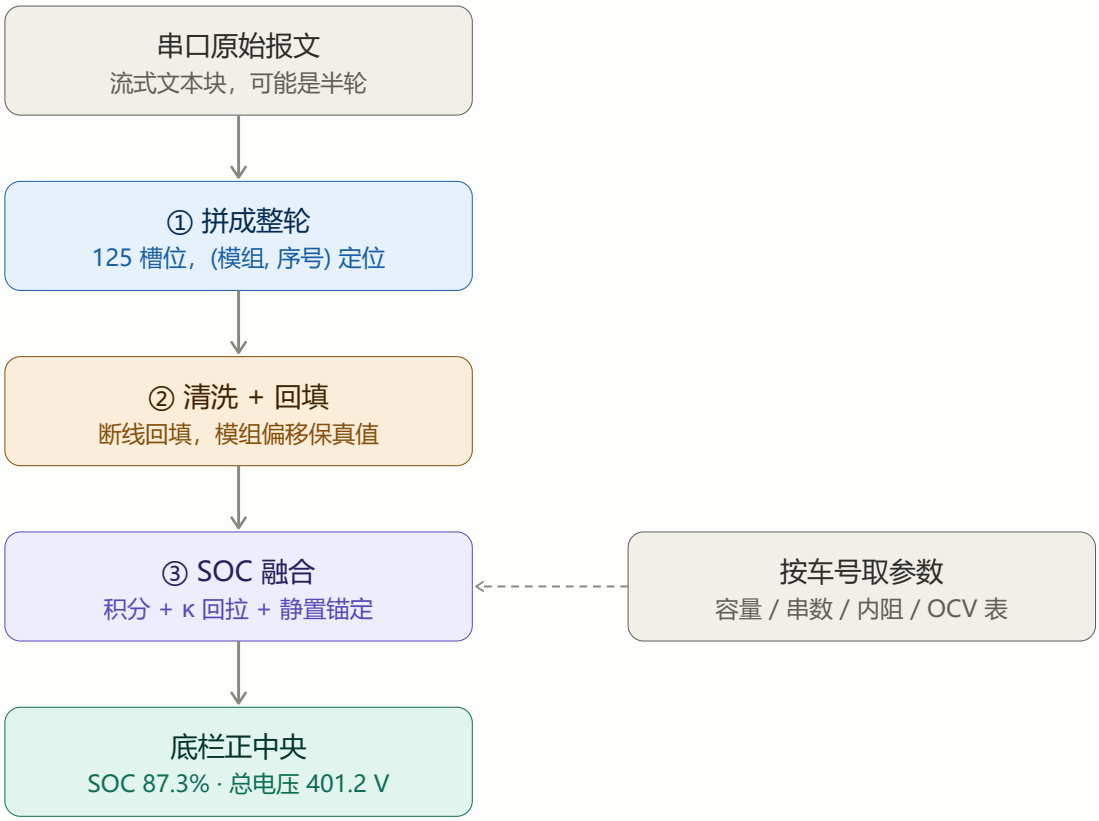


图 2：SOC 算法数据流（拼整轮 → 清洗回填 → 融合 → 输出）

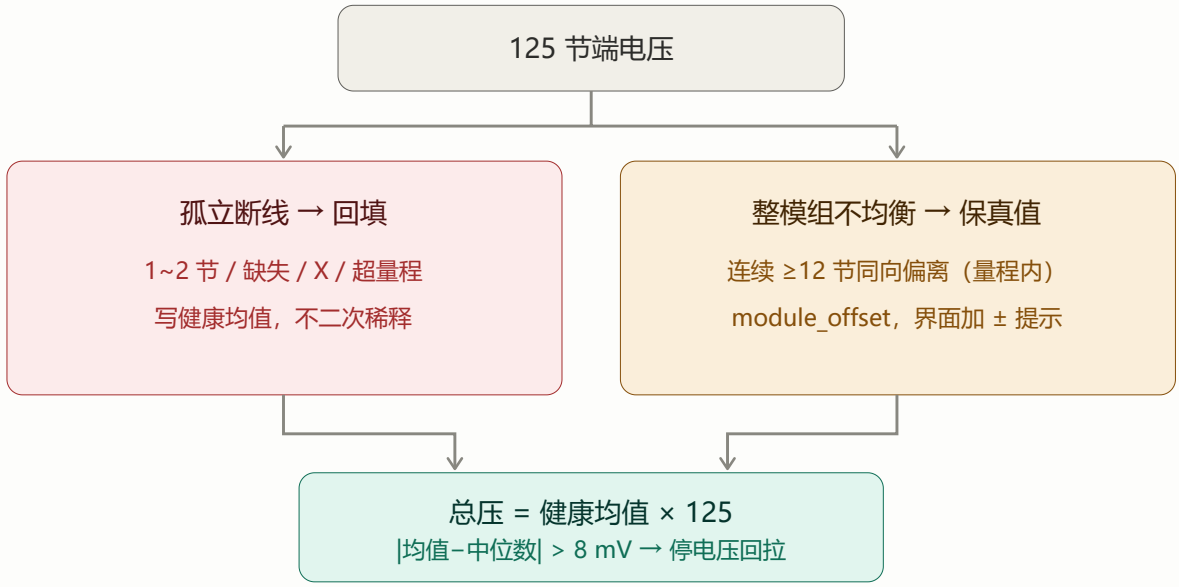
4.0 第 0 步：拼成整轮

把分帧的电压/温度按 `(模组号, 序号)` 重组成 125 槽位。**序号在每个模组内部从 01 重排**，只按序号存会把模组 2 覆盖模组 1，最后只剩 25 节 —— 这是最容易踩的坑。

```
idx = 模组号(0基) × cells_per_module(25) + 序号(1基) − 1     # 0..124
```

4.1 第 1 步：清洗（区分"断线"和"整模组不均衡"）

```
① 量程筛：只留 2500 ≤ v ≤ 4600 mV，其余 flag = None（必然回填）
② 稳健离群门槛：
   median = median(healthy)
   MAD    = median(|v − median|)
   IQR    = q3 − q1
   limit  = max(5 × 1.4826 × MAD, 20 mV, 3 × IQR)
   越界 → flag = +1 / −1（记录方向）
③ 连续段长度判据（兜底）：
   run_min = max(2, ⌊25 × 0.5⌋) = 12
   连续同号离群 ≥ 12 节 → 整模组不均衡，保真值（module_offset = True）
   连续同号离群 <  12 节 → 孤立断线，回填
④ 均值 / 中位数差：
   accepted = 所有保真值
   mean = Σaccepted / len(accepted)
   gap  = |mean − median(accepted)| ；  gap > 8 mV → residual = True（本轮停电压回拉）
⑤ 回填：所有 flag=None / 孤立离群 一律写 mean
   均值只算一次，回填值不参与二次计算
```



关键：超量程（0 mV）一律回填，绝不会被当成模组偏移保留

图 3：清洗环节区分断线与整模组不均衡

两条非对称规则（⛔ 别改坏）：

- 超量程（0 mV）→ 必然回填。整个模组采样线断了、25 节全读 0，不会被当成"模组偏移"保留 —— 否则总压塌成 320 V、SOC 显示 0%，把接线故障伪装成"没电"。
- `run_min = 12` 是故意留的悬崖：11 节连续 +100 mV 仍会回填。⛔ 别为了"11 节也报"调低它，真机每轮有 8 个坏点，门槛一降就可能把正常格子误判成模组偏移从而保留错值。

4.2 第 2 步：安时积分（骨架，短期准）

```
TEXT
soc ← soc − i_eff × dt / 3600 / Q_NOM_AH × 100
# i_eff 已扣零漂、已按 i_sign 归一（正 = 放电）；Q_NOM_AH = 12.041
# dt 异常（<0 或 > 10 s）→ dt = 0（当"没有这段时间"，防一次拉爆）
```

4.3 第 3 步：κ 加权电压回拉（长期准，慢纠偏）

```
TEXT
ocv_v   = (mean_mv + i_eff × R0_MOHM) / 1000          # IR 补偿
soc_ocv = soc_from_ocv(ocv_v)                          # 查第 5 章 OCV 表，线性插值，表外不外推
k       = κ(soc) × 0.02                                # κ 见下；K_VOLT = 0.02
soc    += k × (soc_ocv − soc)
# ⛔ residual = True 时跳过这一步（均值可能还被没剔干净的断线拽着）
```

κ 是"此刻电压到底含多少 SOC 信息"的权重：平台段（10~30%）斜率只有 4~10 mV/%，硬纠只会放大噪声，给 0.1；两端斜率大给 1.0。

```
TEXT
κ(soc) = clamp( slope_OCV(soc) / 10.0 , 0.1 , 1.0 )
```

4.4 第 4 步：静置软锚定（唯一的强校正）

四个条件**同时**满足才触发：

条件	门限
电流小	`
电压稳	120 s 窗口内均值极差 `≤ 2.0 mV`
时间长	稳定状态持续 `≥ 120 s`
温度合适	`0 ~ 60 °C`

```
TEXT
delta = clamp(soc_ocv − soc, −15%, +15%)
soc   += 0.3 × delta          # 增益 0.3、封顶 ±15%：小步纠偏，不硬拉到查表值
```

4.5 第 5 步：电流零漂在线学习

```TEXT
if 接触器全断(三个 bit 全 0) and BMS_OK:
    if |raw| ≤ 20 A:
        i_bias += 0.25 × (raw − i_bias)     # 一阶低通，读数就是零漂
        i_bias  = clamp(i_bias, ±5 A)
    i_eff = 0
else:
    i_eff = (raw − i_bias) × i_sign         # i_sign = +1（正=放电）
```

唯一能可靠学到零漂的时机就是母线全断（真机实测收敛到约 1.41 A）。`|raw| > 200 A` 的帧整帧作废（不积分、不学零漂）。

4.6 第 6 步：工况判别（决定突出谁 + C125 标记）

按信号可靠度分层（不是随便排）：

层	判据	结论
0	`mean_mv ≤ 0`	`unknown`
1	接触器全断	`open` **（未接负载）**
2	`BMS OK == False`	`unknown`
3	电流有效且 `	i_eff
4	电流缺/轨值/太小	用单体电压长窗口趋势兜底（60 s 窗口，变化 ≥ 3 mV 判充/放）
5	其余	`idle`（带 30 s 滞回）

⚠️ 接触器检查放在 BMS 之前：真机上 `BMS Error` 与"母线断开"会同时出现，而"母线已断"是硬件级事实，对用户判断"有没有接负载"更有用。

---

5. E74 参数一页纸（直接抄进代码）

5.1 电池参数

常量	值	来源
`Q_NOM_AH`	**12.041** Ah	1C 恒流放电累计容量
`N_CELLS`	**125**	5 模组 × 25
`CELLS_PER_MODULE`	**25**	
`R0_MOHM`	**6.8** mΩ	1C 放电 IR 压降 ≈ 82 mV
`I_SIGN`	**+1.0**	设备读数为正 = 放电
化学体系	NCM 高电压三元锂	

5.2 阈值（全部实测，不是拍的）

类别	常量	值
清洗	`CELL_MIN_MV` / `CELL_MAX_MV`	2500 / 4600
清洗	`MAD_K` / `MAD_FLOOR_MV` / `IQR_K`	5.0 / 20.0 / 3.0
清洗	`MODULE_RUN_RATIO` / `MEAN_MED_MAX_MV` / `MIN_HEALTHY_RATIO`	0.5 / 8.0 / 0.6
融合	`K_VOLT` / `KAPPA_LO` / `KAPPA_HI` / `KAPPA_SLOPE_REF`	0.02 / 0.1 / 1.0 / 10.0
静置	`REST_GAIN` / `REST_CAP_PCT` / `REST_I_MAX_A`	0.3 / 15.0 / 1.2
静置	`REST_WINDOW_S` / `REST_STABLE_MV` / `REST_T_MIN_C` / `REST_T_MAX_C`	120 / 2.0 / 0.0 / 60.0
时序	`DT_MAX_S` / `REINIT_GAP_S`	10.0 / 600.0
电流	`BIAS_GAIN` / `BIAS_LIMIT_A` / `BIAS_SAMPLE_MAX_A` / `I_LIMIT_A`	0.25 / 5.0 / 20.0 / 200.0
报文	`STICKY_ROUNDS`	10

5.3 OCV–SOC 表（26 点，线性插值，表外不外推）

SOC %	OCV V	SOC %	OCV V	SOC %	OCV V	SOC %	OCV V
0	3.0818	15	3.6908	40	3.8208	70	4.1400
2	3.4516	20	3.7128	45	3.8595	75	4.2066
4	3.5595	25	3.7337	50	3.9038	80	4.2695
6	3.6099	30	3.7582	55	3.9546	85	4.3236
8	3.6385	35	3.7872	60	4.0118	90	4.3672
10	3.6581			65	4.0744	95	4.4016
						97	4.4155
						99	4.4558
						100	4.4981

两端锚点：**SOC 100% ↔ 4.4981 V，SOC 0% ↔ 3.0818 V。** 完整表也在 `soc_est.py` 的 `OCV_TABLE` 里，可直接复制。

> ⛔ 改任何阈值，必须用 119 轮历史日志 + 499 轮会话库重新标定。⚠️ 现成车 `A08-2026` 是 `20/20/20/20/26` 不规则模组，扁平索引不成立，会显示 `--`，属已知边界不是 bug。

---

6. ECU 输出给 C125（输出数据契约）

ECU 把算好的 SOC 经 CAN 发给 C125。建议帧：

帧 ID	DLC	字节	字段	类型	单位 / 范围
`0x700`	8	0	SOC 主值	uint8	0~100（×1 %）
		1-2	**SOC min**	uint16	×0.1 %（0~1000）
		3-4	**SOC max**	uint16	×0.1 %（0~1000）
		5-6	总压	uint16	×0.1 V
		7	状态字节	uint8	见下
`0x701`	2	0-1	母线电流	int16	×0.1 A，放电为正

状态字节 bit 定义：

bit	含义	bit	含义
0	valid（有效）	4	母线断开（open）
1	充电	5	警告
2	放电	6	弱（weak）
3	静置	7	未初始化

更新率 **2~5 Hz** 即可。

6.1 C125 配置要点

- MoTeC **C125 只认 CAN 总线**，靠「自定义 CAN stream + **DBC 文件**」把帧映射成通道。
- 多字节字段（uint16 / int16）的**字节序选 "Word Swap" = 小端**（低字节在前）。
- 波特率与 BMS/ECU 一致：**500 kbps**（或 1M）。
- 仪表上 **SOC 只显示 min 和 max 两枚**（不再单一中心值）：放电突出 min、充电突出 max、静置/母线断开默认突出 min；工况标记（充电/放电/静置/母线断开）单独发，不占用 SOC 读数位。

建议 DBC 片段（示意）：

```TEXT
BO_ 1792 ECU_SOC: 8 ECU          // 0x700
 SG_ SOC_Main    : 0|8@1+ (1,0)   [0|100]   "%"   C125
 SG_ SOC_Min     : 8|16@1+ (0.1,0) [0|100]   "%"   C125
 SG_ SOC_Max     : 24|16@1+ (0.1,0) [0|100]   "%"   C125
 SG_ Pack_V      : 40|16@1+ (0.1,0) [0|6553] "V"   C125
 SG_ Status      : 56|8@1+  (1,0)   [0|255]   ""    C125

BO_ 1793 ECU_CURRENT: 2 ECU       // 0x701
 SG_ Bus_I       : 0|16@1- (0.1,0) [-3276|3276] "A" C125
```

（`@1+` 表示小端无符号，`@1-` 表示小端有符号；具体通道名按你们 C125 工程习惯取。）

---

7. 联调与验收判据（怎么知道算对了）

7.1 用 SerialAssistant 当离线基准

SerialAssistant 是同一套算法的 **PC 端参考实现**（纯 Python、无 Qt 依赖、可单测）。把 BMS 的串口日志（`Voltage1: … I= …` 文本）喂给它，得到一条 SOC 曲线；ECU 算法喂**相同输入**应得到一致结果。这是最低成本的"对拍"方式。

7.2 验收清单（逐条打勾）

- [ ] **冷启动**：上电第一次有数据时按 OCV 查表定初值，不应跳变或归零。
- [ ] **静置锚定**：车停、母线断开、静置 ≥ 2 min 后 SOC 稳定，与 OCV 查表值偏差 < ±2%。
- [ ] **单调性**：放电时 SOC 单调下降、充电时单调上升；全过程中 SOC 始终在 0~100，不越界。
- [ ] **母线断开**：接触器全断时电流判 0、工况标记 `open(母线断开)`，且不拿 −250 A 轨值去积分。
- [ ] **断线不塌**：模拟 25 节全 0 mV（采样线断），总压应回填健康均值（≈ 400 V），SOC 不归零。
- [ ] **量程内偏移保留**：整模组 ±100 mV 同向偏移应被识别为 `module_offset` 并保真值（不回填），界面/数据加 `±`。
- [ ] **C125 显示**：DBC 配好后，C125 上 SOC min/max、总压、电流、工况标记与 ECU 输出一致，字节序正确。

7.3 期望精度参考（PC 端实测）

指标	数值
闭环回归（1C 曲线重放，51 个 2% 台阶）	最大偏差 **1.38%**
静置轮与 OCV 查表一致性	494 轮最大偏差 **0.09%**
平台内读数稳定性（10 mV 细桶）	跨度 ≤ **0.11%**
零漂收敛	约 **1.41 A**
SOC 越界次数	**0**

> ⚠️ 明确边界：E74 没有任何一次"带真实 SOC 标签"的放电记录（无实验室库仑计、无分容报告），所以上面是**自洽性/一致性**指标，不是绝对精度。要拿绝对精度，需做一次带库仑计的满充→满放标定重新拟合 OCV 表。

---

8. 用 SerialAssistant 当验证工具

参考实现与验证脚本位置（都在项目仓库里）：

路径	说明
`soc_est.py`	算法 / 清洗 / 极值 / 工况判别 / OCV 查表 / 格式化（**纯 Python，无 Qt**）
`soc_feed.py`	报文流 → 整轮（纯 Python）
`soc_profiles.py`	按车号取参数（`BUILTIN_PACKS["E74"]`）
`devtools/_verify_soc.py`	141 项单测（查表/清洗/估算/报文/界面/工况）
`devtools/_soc_backtest.py`	真机回测（119 轮日志 + 499 轮会话库）
`devtools/_verify_soc_ekf.py`	四状态 EKF 数值自测 14 项

对拍流程：把 BMS 串口日志导出成文本 → 用 `_verify_soc.py` / `_soc_backtest.py` 跑出基准 SOC 曲线 → ECU 侧用相同输入跑你们的移植版 → 对比两条曲线偏差是否在 §7.3 量级。

附录 A：改代码 / 调参前必读

1. 改任何**清洗阈值**，必须用 119 轮历史日志 + 499 轮会话库重新标定，确认 `residual` / `module_offset` 误报仍为 0 轮。
2. **别动两条非对称规则**：超量程必然回填、只有量程内偏移才能判 `module_offset`；`run_min = 12` 是故意留的悬崖。
3. `-250 A` **轨值必须整帧作废**，不钳位、不学零漂。
4. 电流方向若装反（开车 SOC 反涨），只改 `I_SIGN` 这一个常量即可。
5. 新增一台车：往 `soc_profiles.BUILTIN_PACKS` 加一条（容量/串数/内阻/OCV 表），估算链路一行都不用改。
6. 只支持**等长模组**；不规则模组（如 `A08-2026`）显示 `--`。
7. 版本号与发布由用户定；本算法移植到 ECU 不涉及 SerialAssistant 的发布流程。

附录 B：参考文献

1. G. L. Plett, "Extended Kalman filtering for battery management systems…" *J. Power Sources* 134 (2004) 252–292.（安时积分 + 电压反馈框架来源）
2. R. Xiong et al., "A review on Li-ion battery SOC estimation", *IEEE Access* 6 (2018).
3. 陈息坤 等，动力电池 SOC 估算，*电工技术学报*，2015（工程实测约 3% 误差）。
4. 林成涛 等，OCV–SOC 标定方法，2004。

> 选型结论：Plett 的「安时积分 + 电压反馈」框架是对的，但完整 EKF 在 E74 的抖动采样 + 零漂输入下协方差会发散，故落地为「固定增益 + κ 可信度加权 + 静置软锚定」。ECU 上若算力充裕且想上 EKF，参见 `soc_ekf.py`（四状态二阶 RC-PNGV，纯 Python、无 numpy 依赖）。
